Problem 1:

Given:
The quadratic function is defined as:

$$f(x) = \frac{1}{2}x^\top Qx + q^\top x + c$$

For any quadratic function of this form, the Hessian matrix is constant and exactly equal to Q, so $\nabla^2 f(x) = Q$.

To determine the smoothness and strong convexity parameters, we evaluate the eigenvalues of Q by finding the characteristic polynomial, $\det(Q - \lambda I) = 0$:

$$\det \begin{bmatrix} 13-\lambda & 12 & -2 \\ 12 & 17-\lambda & 6 \\ -2 & 6 & 12-\lambda \end{bmatrix} = 0$$

Expanding the determinant yields $-\lambda^3 + c_2\lambda^2 - c_1\lambda + c_0 = 0$, where:

- c_2 **(Trace of Q):** $13 + 17 + 12 = 42$
- c_1 **(Sum of principal minors):** $(17 \cdot 12 - 36) + (13 \cdot 12 - 4) + (13 \cdot 17 - 144) = 397$
- c_0 **(Determinant of Q):** $13(168) - 12(156) - 2(106) = 100$

This gives the characteristic polynomial:

$$\lambda^3 - 42\lambda^2 + 397\lambda - 100 = 0$$

Solving numerically, the eigenvalues are:

$$\lambda_1 \approx 0.258, \quad \lambda_2 \approx 13.856, \quad \lambda_3 \approx 27.886$$

(a) Smoothness parameter:
Answer: Yes, $L \approx 27.886$.
A function is L-smooth if $\lambda_{\max}(\nabla^2 f(x)) \leq L$. Since Q is constant with finite eigenvalues, $f(x)$ is smooth with parameter $L = \lambda_{\max}(Q) \approx 27.886$.

(b) Strong convexity parameter:
Answer: Yes, $\mu \approx 0.258$.
A function is μ-strongly convex if $\nabla^2 f(x) \succeq \mu I$ for $\mu > 0$. Since all eigenvalues of Q are strictly positive, Q is positive definite. The parameter is $\mu = \lambda_{\min}(Q) \approx 0.258$.

Problem 2:

Mathematical Setup:
For $f(x) = \frac{1}{2}x^\top Qx + q^\top x + c$, the gradient is $\nabla f(x) = Qx + q$. Since $Q \succ 0$, the unique global minimum x^* satisfies $Qx^* + q = 0 \implies x^* = -Q^{-1}q$. The optimal value is $p^* = f(x^*)$.

(a) Fixed Step Size Gradient Descent
Using the smoothness parameter $L = \lambda_{\max}(Q) \approx 27.886$, the update rule is:

$$x_{k+1} = x_k - t\nabla f(x_k) \quad \text{where} \quad t = \frac{1}{L}$$

(b) Exact Line Search Gradient Descent
We find t_k that minimizes $\phi(t) = f(x_k - tg_k)$, where $g_k = \nabla f(x_k)$. Setting $\phi'(t) = 0$:

$$-g_k^\top (Qx_k + q) + tg_k^\top Qg_k = 0 \implies t_k = \frac{g_k^\top g_k}{g_k^\top Qg_k}$$

The update rule is $x_{k+1} = x_k - t_k g_k$.

Results:
The gradient descent algorithm was implemented starting from $x_0 = \mathbf{0}$. As shown below, exact line search converges faster than the fixed step size approach.

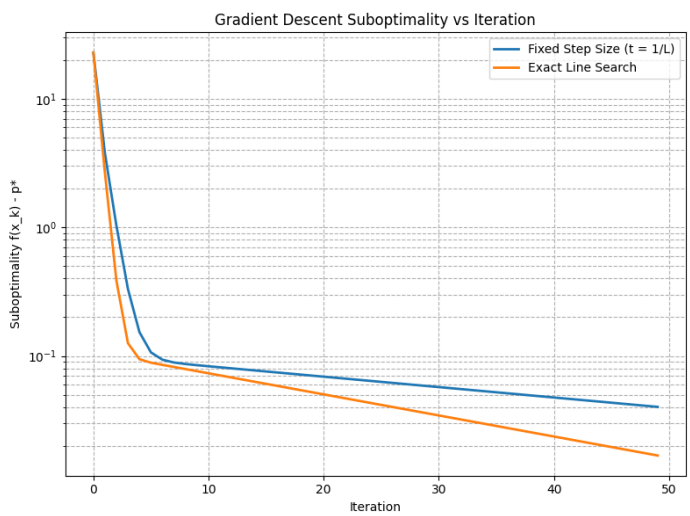

Figure 1: Suboptimality $f(x_k) - p^*$ vs. Iteration k for Gradient Descent

Problem 3:

Proof:
Let $t > 0$ be an arbitrary fixed step size. Consider the 1D function $f(x) = \frac{L}{2}x^2$ for some $L > 0$.

- **Smoothness/Convexity:** $\nabla^2 f(x) = L$. The function is L-smooth and strongly convex ($\mu = L$).
- **Global Minimum:** $x^* = 0$.

The gradient descent update is:

$$x_{k+1} = x_k - t\nabla f(x_k) = x_k - t(Lx_k) = (1 - tL)x_k$$

This linear recurrence yields $x_k = (1 - tL)^k x_0$. For convergence from any $x_0 \neq 0$, we require $|1 - tL| < 1$, which implies $L < \frac{2}{t}$.

If we intentionally choose $L = \frac{3}{t}$ (which satisfies strong convexity since $L > 0$), the update becomes:

$$x_{k+1} = (1 - 3)x_k = -2x_k \implies x_k = (-2)^k x_0$$

For any $x_0 \neq 0$, $|x_k| \to \infty$ as $k \to \infty$. The algorithm diverges. Thus, no single fixed step size t can guarantee convergence for *all* strongly convex functions. ■

Problem 4:

(a) If f_i are convex, show $f(x) := \sup_i f_i(x)$ is convex.
Proof:
Let $\theta \in [0, 1]$. By the convexity of f_i:

$$f_i(\theta x + (1 - \theta)y) \leq \theta f_i(x) + (1 - \theta)f_i(y) \quad \text{for all } i$$

Since $f_i(x) \leq f(x)$ and $f_i(y) \leq f(y)$:

$$f_i(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y) \quad \text{for all } i$$

Taking the supremum over i yields:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

Thus, $f(x)$ is convex. ■

(b) Show $\lambda_{\max}(M)$ is convex in M. Is the max magnitude eigenvalue convex?
Proof:
By the Rayleigh-Ritz theorem, $\lambda_{\max}(M) = \sup_{\|v\|_2=1} v^\top M v$. For any fixed v, $g_v(M) = v^\top M v$ is linear (and thus convex) in M. Since $\lambda_{\max}(M)$ is the supremum of a family of convex functions, it is convex by part (a).

Maximum Magnitude: Yes, it is convex. The largest magnitude eigenvalue is the spectral norm $\|M\|_2 = \max(\lambda_{\max}(M), -\lambda_{\min}(M))$. Since $-\lambda_{\min}(M) = \lambda_{\max}(-M)$, this term is also convex (composition of a convex and affine function). The maximum of two convex functions is convex. ■

(c) Show the weighted shortest path $f(w)$ is a concave function of w.
Proof:
Let $\mathcal{P}$ be the set of all paths from a to b. The weight of path $p \in \mathcal{P}$ is $W_p(w) = c_p^\top w$, where c_p is an indicator vector for the edges in p.

For any fixed p, $W_p(w)$ is linear and therefore concave. The shortest path is $f(w) = \min_{p \in \mathcal{P}} W_p(w)$. Just as the supremum of convex functions is convex, the infimum (minimum) of a family of concave functions is concave. Thus, $f(w)$ is concave. ■

Appendix: Python Implementation

```python
import numpy as np
import matplotlib.pyplot as plt

Q = np.array([[13, 12, -2], [12, 17, 6], [-2, 6, 12]])
q = np.array([-22, -14.5, 13])
c = -1

x_star = np.linalg.solve(Q, -q)
p_star = 0.5 * x_star.T @ Q @ x_star + q.T @ x_star + c

def f(x):
    return 0.5 * x.T @ Q @ x + q.T @ x + c

def grad_f(x):
    return Q @ x + q

L = max(np.linalg.eigvals(Q))
t_fixed = 1 / L
n_iters = 50
x0 = np.zeros(3)

x = x0.copy()
subopt_fixed = []
for _ in range(n_iters):
    subopt_fixed.append(f(x) - p_star)
    x = x - t_fixed * grad_f(x)

x = x0.copy()
subopt_ls = []
for _ in range(n_iters):
    subopt_ls.append(f(x) - p_star)
    g = grad_f(x)
    t_ls = (g.T @ g) / (g.T @ Q @ g)
    x = x - t_ls * g

# Plotting
plt.figure(figsize=(8, 6))
plt.plot(subopt_fixed, label='Fixed Step Size (t = 1/L)', linewidth=2)
plt.plot(subopt_ls, label='Exact Line Search', linewidth=2)
plt.yscale('log')
plt.xlabel('Iteration')
plt.ylabel('Suboptimality f(x_k) - p*')
plt.title('Gradient Descent Suboptimality vs Iteration')
plt.legend()
plt.grid(True, which="both", ls="--")
plt.tight_layout()
```